EXECUTIVE SUMMARY

Software Requirements Specification (SRS)

Project Title: Jewelry Inventory Auditor Software

Version: 1.0

Date: May 13, 2025

1. Introduction

- **1.1 Purpose:** This document outlines the requirements for a software application designed to streamline the jewelry inventory auditing process across multiple sales counters. The software will enable auditors to efficiently tally inventory against a morning baseline, identify missing items, and generate audit reports.
- **1.2 Scope:** This project's initial phase will focus on developing a system with a backend server (hosted on the client's admin PC) and a frontend application (to be used by auditors on up to 12 PCs). A basic Super Admin dashboard for centralized report viewing is also included.

- **1.3 Intended Audience:** This document is intended for the client (to understand the proposed solution), the development team (as a guide for implementation), and any stakeholders involved in the project.

2. Overall Description

- **2.1 Product Perspective:** The software will be a standalone application integrated into the client's existing local network. It will interact with data provided through XLSX/CSV files and barcode scanners connected to the auditor PCs.
- **2.2 User Classes and Characteristics:**
    - **Auditor:** Employees are responsible for performing the daily inventory audits at each sales counter. They will need to log in, upload morning inventory, scan items, and generate reports. They are expected to have basic computer literacy.
    - **Super Admin:** The client or a designated manager who will oversee the audit process across all locations. They will use a web-based dashboard to view generated audit reports.
- **2.3 Operating Environment:**
    - **Auditor PCs:** Standard Windows-based PCs with web browser access (or a locally installed application). **USB ports for barcode scanners.
    - **Admin PC (Server):** Windows-based PC to host the backend application and potentially the database. Local network connectivity to all auditor PCs.

- **2.4 Design and Implementation Constraints:**
    - **Development Technology:** Flask (backend), React (frontend), and either SQLite or PostgreSQL (database).
    - **Local Network Operation:** The system must function primarily within the client's local network, or on cloud whichever is decided.
    - **Barcode Scanning:** The system must seamlessly integrate with standard USB barcode scanners.

3. Specific Requirements

- **3.1 Functional Requirements:**

    - **3.1.1 Auditor Login:** Auditors must be able to log into the frontend application using their assigned credentials.
    - **3.1.2 Counter & Salesman Selection:** After login, auditors will select the specific sales counter and the assigned salesman for the audit.
    - **3.1.3 Morning Inventory Upload:** Auditors must be able to upload a CSV file containing the in-stock inventory for the selected counter. The CSV should include columns for barcode, article name, and category.
    - **3.1.4 Inventory Display:** The frontend will display the morning inventory data (barcode, name, category) on the left side of the screen.

- **3.1.5 Barcode Scanning:** The software must capture barcode data when an item is scanned using a USB barcode scanner.
- **3.1.6 Real-time Matching:** Upon scanning a barcode, the system will search for a match in the morning inventory. If found, the item (or its barcode) will visually move or be indicated as "scanned" (e.g., moved to the right side of the screen).
- **3.1.7 Missing Items Identification:** Items present in the morning inventory but not scanned will be clearly identified as potentially missing.
- **3.1.8 Report Generation:** Auditors can trigger the generation of an audit report.
- **3.1.9 Audit Report Content:** The generated report will include:
    - Auditor ID
    - Date and Time of Audit
    - Salesman Name
    - Counter Name
    - Total number of items in the morning inventory
    - Total number of items scanned
    - List of missing items (barcode, name, category)
    - Summarized line for the previous report.
- **3.1.10 Report Download/Print:** Auditors should be able to download or print the generated report.
- **3.1.11 Super Admin Login:** The Super Admin must be able to log into a web-based dashboard.

- **3.1.12 Centralized Report Viewing:** The Super Admin dashboard will display a list of all audit reports generated by all auditors, with details like auditor ID, date, and counter.
- **3.1.13 Report Approval/Disapproval (Basic):** The Super Admin should have a basic mechanism to mark reports as “Approved” or “Raise Query”.

- **3.2 Non-Functional Requirements:**

  - **3.2.1 Performance:** The system should provide a responsive experience for barcode scanning and data display.
  - **3.2.2 Reliability:** The system should be stable and function without frequent errors.
  - **3.2.3 Usability:** The auditor interface should be intuitive and easy to use for efficient daily audits.
  - **3.2.4 Security:** Basic user authentication for auditors and the Super Admin.

4. Future Enhancements (Post development and delivery)

- Integration with a central inventory management system.
- Advanced reporting and analytics for the Super Admin.
- User role and permission management.

5. Data Requirements

- **Inventory Data:** CSV file with columns: Barcode (Unique Identifier), Article Name (Text), Category (Text).
- **Audit Logs:** Records of audit sessions, including auditor, counter, time, and report details.
- **User Credentials:** Secure storage of auditor and Super Admin login information.

7. Potential Issues

- **Barcode scanning:** Developer needs to get the entire overview and a demo for how the barcode device works in the client's shops.
  We need to address this issue as this will actively impact the development process, method and stratergy.
- **Deployment:** As already mentioned by client**,** they own a server that can be used as the parent device to store the backend and the admin panel. Rest all of the PC’s will be used as the front end app that can be used by the auditors after logging in into them. The developer team will need a brief on the internal network setup on the clients's system to deploy the backend and frontend separately and to have seamless data transactions.